NP Problem

สุนทรี คุ้มไพโรจน์

การแก้ปัญหา

- ต้องการคำตอบที่ดีที่สุด (Optimal solution)
  หรือ คำตอบที่เป็นไปได้ (Feasible solution)
- เช่น ปัญหา traveling salesman ที่ต้องการเดินทางให้ครบทุกเมือง โดยมีระยะทางรวมสั้นที่สุด
  - ระยะเวลาที่ใช้ในการหาคำตอบที่ดีที่สุด $O(n!)$
  - ระยะเวลาที่ใช้ในการหาคำตอบที่เป็นไปได้ $O(n^a)$
- ระยะเวลาที่ใช้ในการหาคำตอบที่ดีที่สุดของแต่ละปัญหาขึ้นกับลักษณะธรรมชาติของปัญหานั้นๆ

อัลกอริทึมที่มีและไม่มีประสิทธิภาพ

- **Search**
  - O(n) : sequential search → ช้า
  - O(log n) : binary search → เร็ว
- **Sorting**
  - $O(n^2)$: bubble sort, insertion sort → ช้า
  - O(n log n) : merge sort, heap sort → เร็ว
- **Longest Common Subsequence**
  - $O(2^n)$: brute force → ช้า
  - $O(n^2)$: dynamic programming → เร็ว

อะไรจะเป็นตัวบอกว่าอัลกอริทึมมีประสิทธิภาพ ?

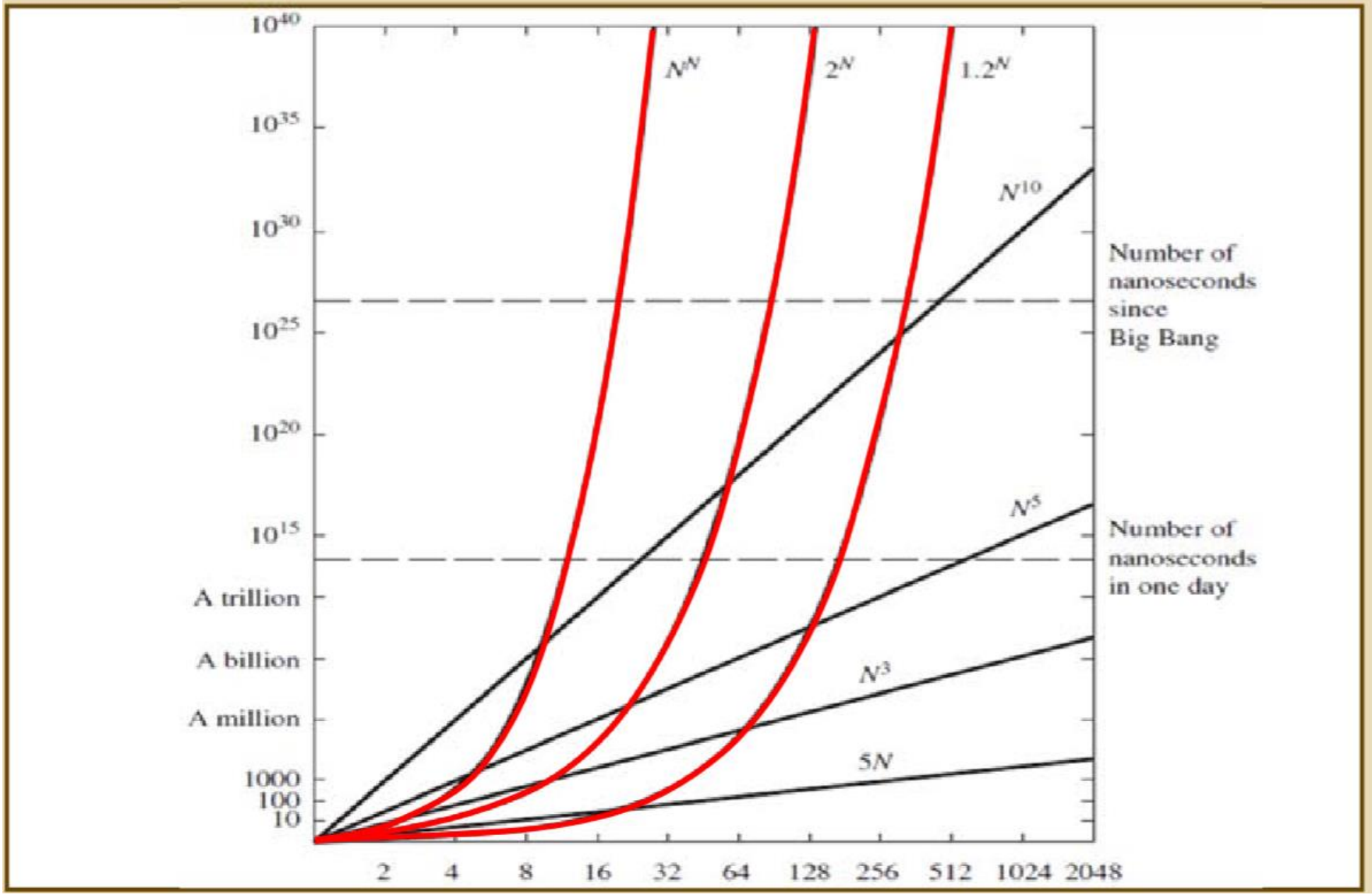
ที่มา เอ็นพีบริบูรณ์ ของ อ.สมชาย ประสิทธิ์จูตระกูล

การขยายค่าของนิพจน์แบบต่าง ๆ

นิพจน์ต่าง ๆ	ค่าของตัวแปร n					
	5	10	20	25	35	50
n	5	10	20	25	35	50
n^2	25	100	400	625	1225	2500
n^3	125	1000	8000	15625	42875	125000
n^4	625	10000	160000	390625	1500625	6250000
2^n	32	1024	1048576	33554432	3.44×10^{10}	1.13×10^{15}
3^n	243	59049	3.49×10^{9}	8.47×10^{11}	5.00×10^{16}	7.18×10^{23}

ที่มา ศ.ดร.ชิดชนก เหลือสินทรัพย์ ,Analysis&design of algorithms

Polynomial vs. Exponential



ที่มา เอ็นพีบริบูรณ์ ของ อ.สมชาย ประสิทธิ์จูตระกูล

การแบ่งกลุ่มปัญหา

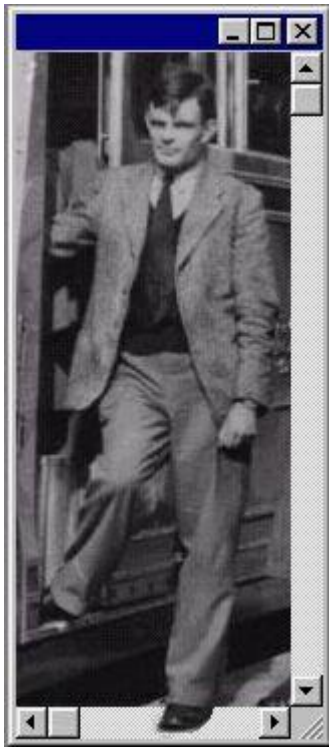
- กลุ่มปัญหา P
  - ปัญหาที่แก้ไขได้ในระยะเวลาที่เป็น polynomial
- กลุ่มปัญหา NP
  - ปัญหาที่แก้ไขโดยวิธี Non-deterministic Polynomial
  - ปัญหาที่อาจแก้ไขได้ในเวลา P โดยวิธีการเดาคำตอบ
  - กำหนดระยะเวลาในการเดาคำตอบในเวลา P

การแบ่งกลุ่มปัญหา

- ปัญหาง่าย (Tractable problem)
  - มีอัลกอริทึมที่มีประสิทธิภาพซึ่งใช้แก้ปัญหานี้ได้
- ปัญหายาก (Intractable problem)
  - ไม่สามารถหาอัลกอริทึมที่มีประสิทธิภาพในการแก้ปัญหา
  - แต่อาจมีอัลกอริทึมที่มีประสิทธิภาพซึ่งแก้ปัญหาเหล่านี้ได้ที่เรายังไม่รู้จักก็ได้

เครื่องคำนวณแบบ Turing

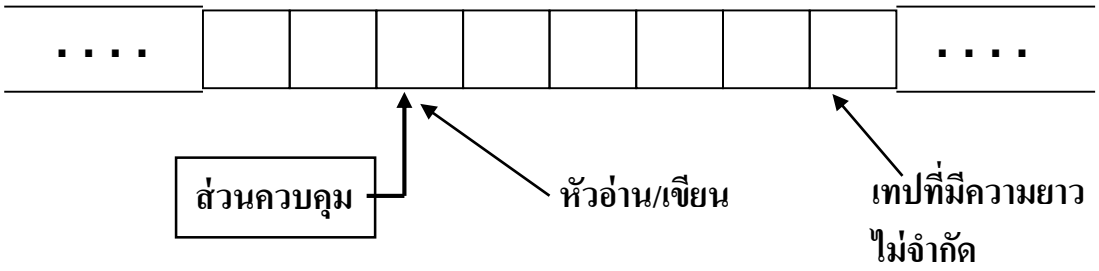
- ปี ค.ศ. 1936 Alan Turing
- แบบจำลองของเครื่องคำนวณที่มีโครงสร้างง่ายๆ
- ใช้แก้ปัญหาส่วนใหญ่ได้
- แต่มีปัญหาบางประเภทที่ไม่ว่าจะใช้เครื่องคำนวณแบบใดก็ตาม ก็ไม่สามารถจะใช้หาคำตอบของปัญหาเหล่านั้นได้ เช่น ปัญหาที่ว่าจะรู้ได้อย่างไรว่าโปรแกรมที่ให้มาจะหยุดทำงาน (the Halting Problem)

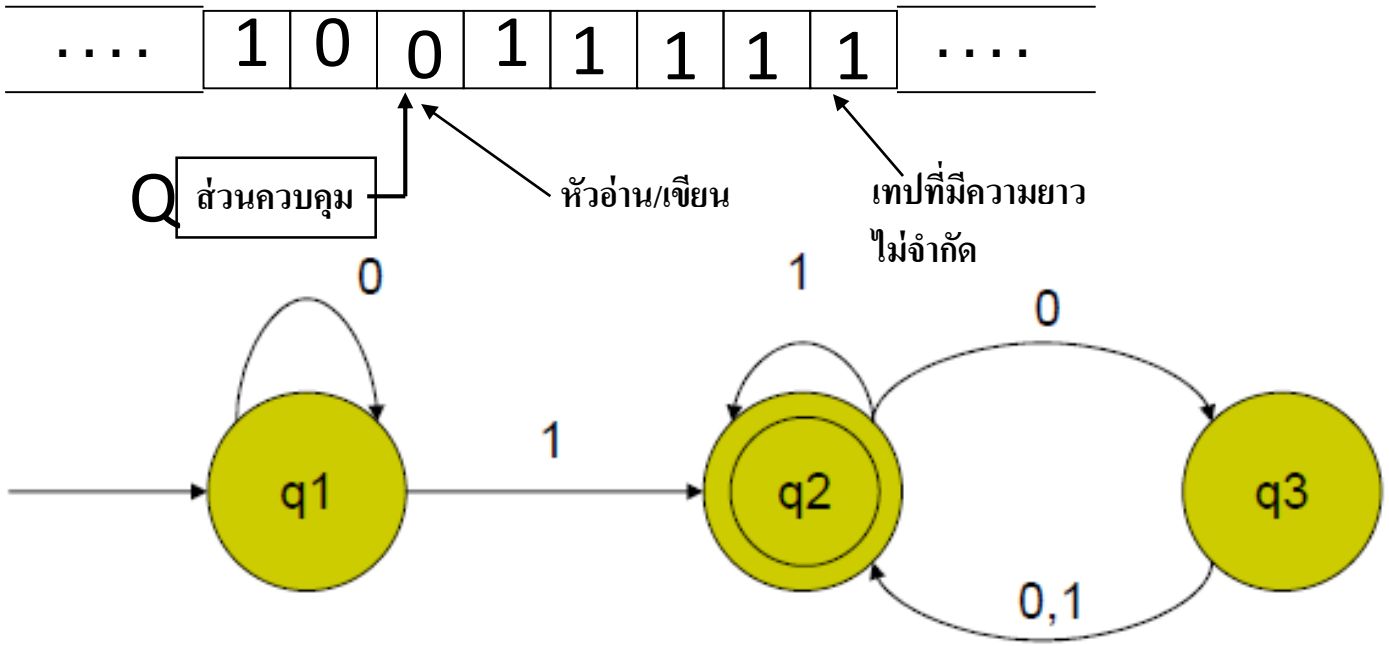


Deterministic Turing Machine (DTM)

ประกอบด้วย

1. **เทปที่มีความยาวไม่จำกัด**
2. **หัวสำหรับอ่านและเขียนตัวอักษรที่อยู่บนเทป**
   - **อ่านตัวอักษรครั้งละหนึ่งตัวอักษร**
   - **สามารถเลื่อนไปมาทางซ้ายหรือขวาได้ หรือไม่เลื่อนเลย**
3. **ส่วนควบคุม**
   - **ตัดสินใจว่า จะให้หัวอ่าน/เขียนบันทึกตัวอักษรอะไรลงไปบนเทป**





We can describe M_1 formally by writing $M_1 = (Q, \Sigma, \delta, q_1, F)$, where

- $Q = \{ q1, q2, q3 \}$
- $\Sigma = \{ 0, 1 \}$
- δ is described as

	0	1
q1	q1	q2
q2	q3	q2
q3	q2	q2

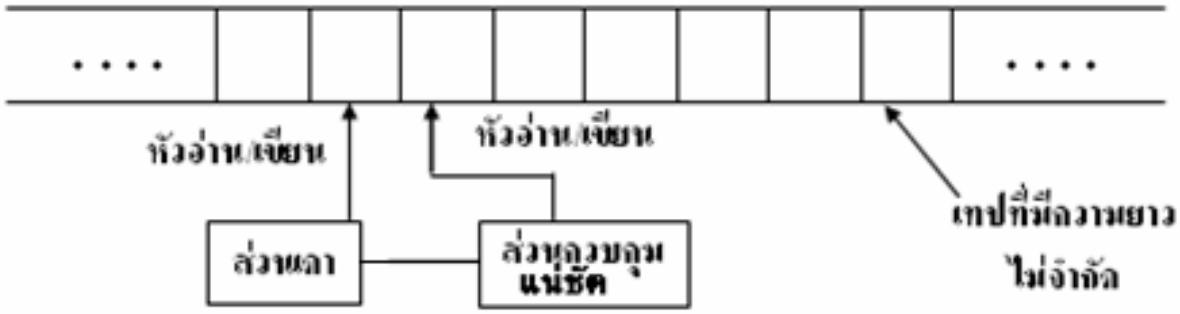
$\delta(q1,0) = q1$ $\quad \delta(q1,1) = q2$
$\delta(q2,0) = q3$ $\quad \delta(q2,1) = q2$
$\delta(q3,0) = q2$ $\quad \delta(q3,1) = q2$

- q1 is the start state, and
- $F = \{ q2 \}$.

Non- deterministic Turing Machine (NDTM)

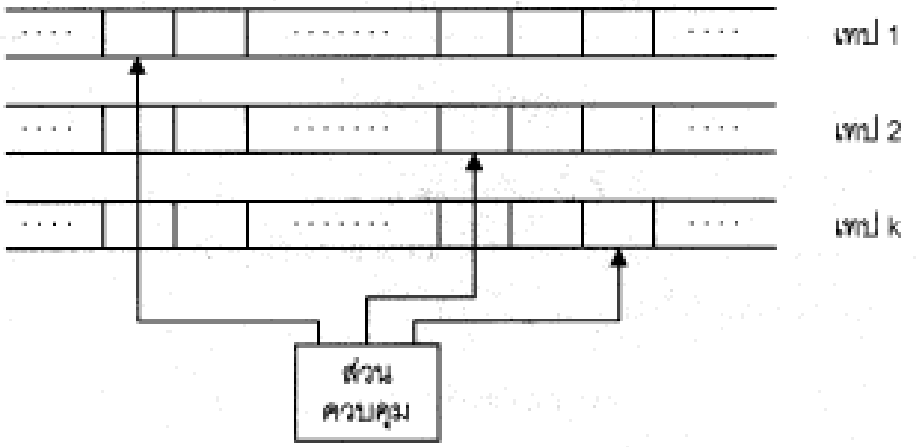
ประกอบด้วย

1. เทปที่มีความยาวไม่จำกัด
2. หัวสำหรับอ่านและเขียนตัวอักษรที่อยู่บนเทป
   - หัวนี้อ่านตัวอักษรครั้งละหนึ่งตัวอักษรและสามารถเลื่อนไปมาทางซ้ายหรือขวาได้ หรือไม่เลื่อนเลย
3. ส่วนควบคุมแน่ชัดและส่วนเดา
   - ตัดสินใจว่า จะให้หัวอ่าน/เขียนบันทึกตัวอักษรอะไรลงไปบนเทป

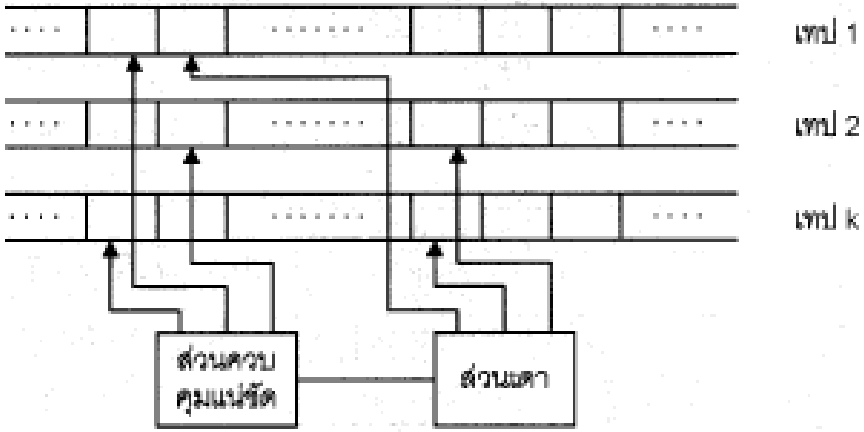


DTM and NDTM with k tapes

DTM with k tapes



NDTM with k tapes



ปัญหายาก

- ปัญหาแจกแจงเซตย่อยทุกกรณี มีความซับซ้อน $O(2^n)$
- ปัญหาย้ายจานหอคอยฮานอย ซึ่งต้องทำการย้าย $2^n - 1$ ครั้ง
- ปัญหาประเภทที่พิสูจน์ได้เลยว่าไม่มีอัลกอริทึมใดๆหาคำตอบได้
  - เช่น Hilbert's Tenth Problem ซึ่งต้องการหาคำตอบของสมการ Diphantine ตัวอย่างเช่น จงหาว่ามีคำตอบจำนวนเต็มของตัวแปร x, y และ z ในสมการ $42x^2y + 34xz = 112$ หรือไม่ ?
- ปัญหา Halting problem ซึ่ง Alan Turing เป็นผู้พิสูจน์ว่าเป็นปัญหาแบบคำนวณไม่ได้ในปี ค.ศ. 1937

ปัญหาการตัดสินใจ

- ปัญหาการตัดสินใจคือปัญหาที่ให้คำตอบว่า
  ใช่/ไม่ใช่ ได้/ไม่ได้ มี/ไม่มี จริง/ไม่จริง มีเพียงสองคำตอบ
- ตัวอย่างของปัญหาประเภทนี้ เช่น
  - Satisfiability (SAT)
  - Hamilton Circuit (HAM)
  - Traveling Salesperson Problem (TSP)

ตัวอย่างปัญหา Satisfiability (SAT)

กำหนดให้ f คือนิพจน์บูลีน (Boolean expression) ซึ่งเขียนในรูป Conjunctive normal form – CNF หรือ Product of sums

- Example clauses:

$$C_1 = x_1 + x_2 + \overline{x}_3, \quad C_2 = \overline{x}_1 + \overline{x}_2 + \overline{x}_3, \quad C_3 = \overline{x}_1 + x_2 + x_3$$

$$f = C_1 C_2 C_3 = (x_1 + x_2 + \overline{x}_3)\ (\overline{x}_1 + \overline{x}_2 + \overline{x}_3)\ (\overline{x}_1 + x_2 + x_3)$$

ถ้ากำหนดให้ตัวแปรมีค่าดังนี้ $x_1=1$, $x_2=0$ และ $x_3=1$

$$C_1 = x_1 + x_2 + \overline{x}_3 \rightarrow (1 + 0 + 0) \rightarrow 1$$

$$C_3 = \overline{x}_1 + \overline{x}_2 + \overline{x}_3 \rightarrow (0 + 1 + 0) \rightarrow 1$$

$$C_2 = \overline{x}_1 + x_2 + x_3 \rightarrow (0 + 0 + 1) \rightarrow 1$$

Therefore $f = 1$

ความพึงพอใจ
(Propositional Satisfiability)

- If มีการกำหนดค่าความจริงให้กับตัวแปรที่ทำให้ประพจน์นั้นเป็น **จริง** ได้ then ประพจน์ประกอบนั้น สามารถทำให้เป็นจริงได้ (**satisfiable**), otherwise ประพจน์ประกอบนั้น เป็น **unsatisfiable.**

Ex: $(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p)$ เป็น true
เมื่อตัวแปรทุกตัวมีค่าความจริงเหมือนกัน

แสดงว่า ประพจน์ประกอบ(compound proposition) นั้น **satisfiable**

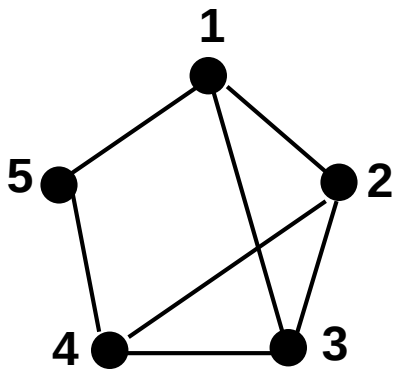
ถ้า p, q, r มีค่าเป็น T แล้ว ประพจน์ข้างบนมีค่า ?

ถ้า p, q, r มีค่าเป็น F แล้ว ประพจน์ข้างบนมีค่า ?

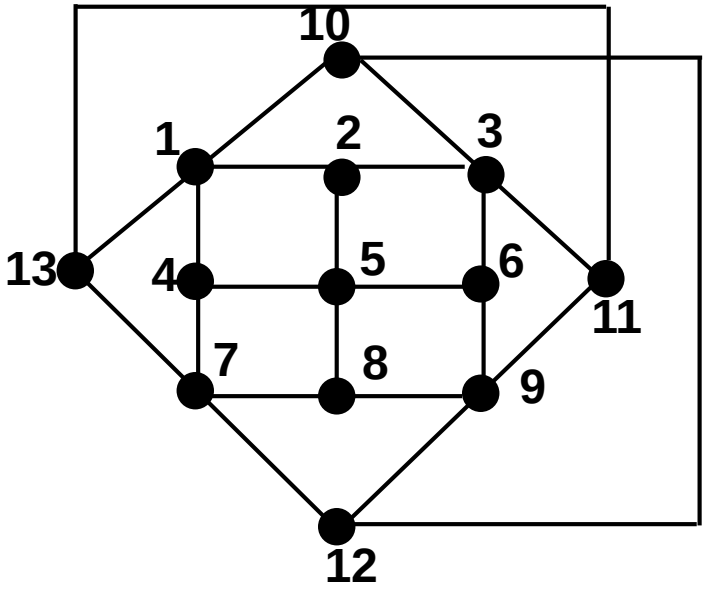
- ประพจน์ประกอบ(compound proposition)นั้น **unsatisfiable** iff นิเสธ(negation)ของประพจน์ประกอบนั้น เป็น **tautology**

ตัวอย่างปัญหา Hamilton Circuit

กราฟนี้มีวงจร Hamilton หรือไม่ (วงจรที่ผ่านจุดยอดทุกจุดๆละหนึ่งครั้ง)



กราฟที่มีวงจร Hamilton



กราฟที่ไม่มีวงจร Hamilton

Traveling Salesperson Problem (TSP)

- กำหนดให้ G คือกราฟ อยากทราบว่ามีวงจรในกราฟนี้ซึ่งผ่านจุดยอด (Vertex) ทุกจุดในกราฟจุดละหนึ่งครั้ง โดยความยาวรวมของวงจรนี้มีค่าไม่เกิน k หรือไม่
- เป็น NP-hard Problem
- วิธีการแก้ปัญหา
  - Exhaustive search
  - Greedy heuristics
  - Approximation problem
  - Local search
    - Simulated annealing

การเปลี่ยนรูปของปัญหา(Reduction)

- กำหนดให้ Q_1 และ Q_2 เป็นปัญหาการตัดสินใจ
  - เปลี่ยนรูปปัญหา Q_1ไปเป็นปัญหา Q_2 (ถ้าเปลี่ยนได้)
  - จากนั้นใช้อัลกอริทึม Q_2 หาคำตอบ ก็จะเป็นคำตอบของตัวอย่างปัญหาของ Q_1 นั้น

Subset sum

Theorem. 3-SAT $\leq_P$ SUBSET-SUM.

Pf. Given an instance Φ of 3-SAT, we construct an instance of SUBSET-SUM that has solution iff Φ is satisfiable.

- ปัญหา 3-Sat สามารถ เปลี่ยนรูป(reduce) เป็นปัญหา subset-sum

3-CNFSAT $\leq_p$ SUBSET-SUM

E.g. $(x_1 \vee x_2 \vee x_3)\ (\bar{x}_1 \vee x_3 \vee x_4)\ (x_1 \vee \bar{x}_2 \vee \bar{x}_3)$

$y_i \in S \Leftrightarrow x_i = \text{True}$

$z_i \in S \Leftrightarrow \bar{x}_i = \text{True}$

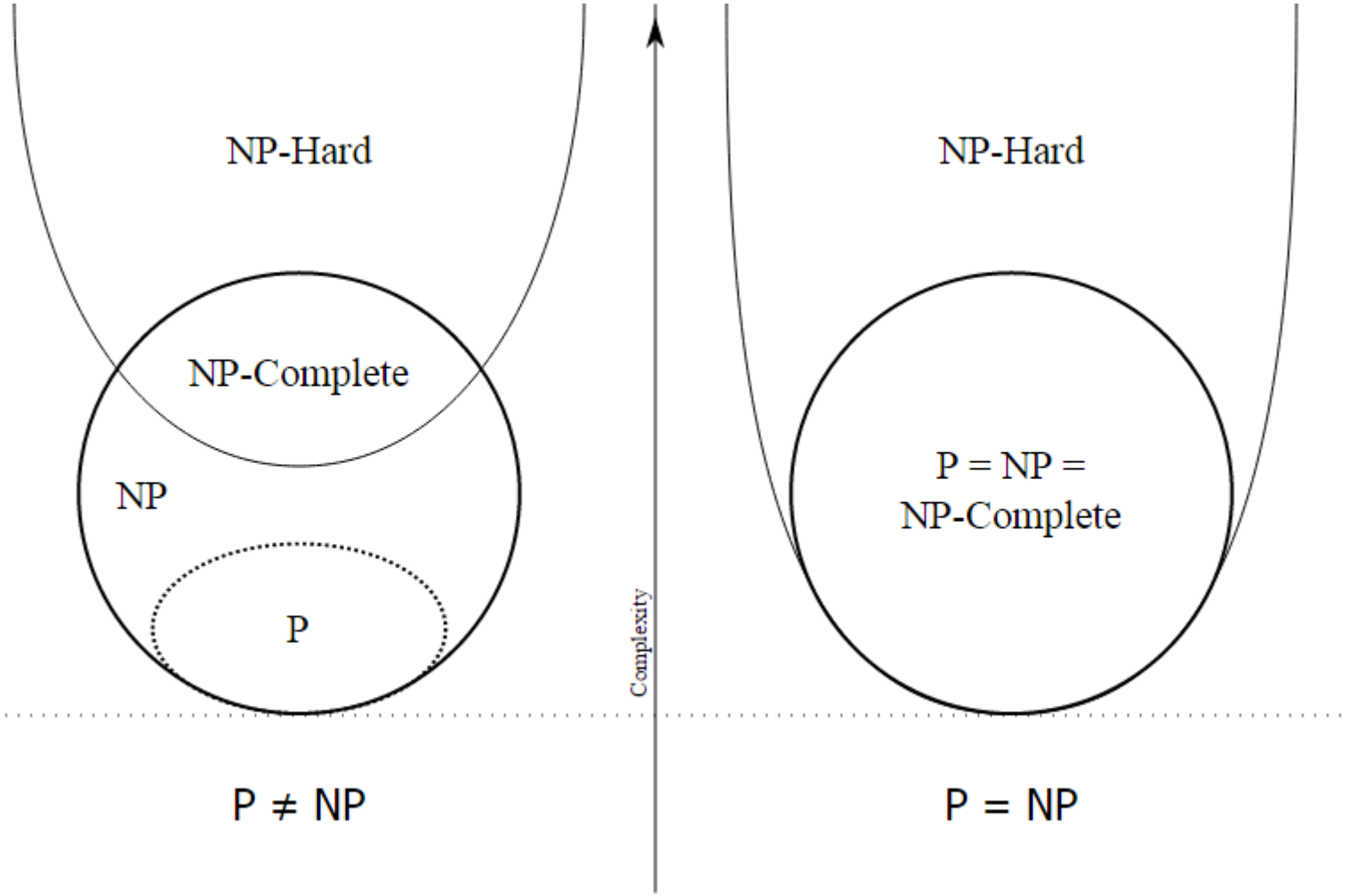
	x_1 (10^6)	x_2 (10^5)	x_3 (10^4)	x_4 (10^3)	C_1 (10^2)	C_2 (10^1)	C_3 (10^0)
y_1	1	0	0	0	1	0	1
z_1	1	0	0	0	0	0	0
y_2		1	0	0	1	0	0
z_2		1	0	0	0	1	1
y_3			1	0	1	1	0
z_3			1	0	0	0	1
y_4				1	0	1	0
z_4				1	0	0	0
t	1	1	1	1	3	3	3

Insert

	C_1 (10^2)	C_2 (10^1)	C_3 (10^0)
g_1	1	0	0
h_1	1	0	0
g_2		1	0
h_2		1	0
g_3			1
h_3			1

ที่มา https://www.youtube.com/watch?v=i8Kt9IBZ8FU

ปัญหา NP Complete



http://th.wikipedia.org/wiki/%E0%B9%80%E0%B8%AD%E0%B9%87%E0%B8%99%E0%B8%9E%E0%B8%B5%E0%B8%9A%E0%B8%A3%E0%B8%B4%E0%B8%9A%E0%B8%B9%E0%B8%A3%E0%B8%93%E0%B9%8C#/media/File:P_np_np-complete_np-hard.svg

ปัญหา NP Complete

- ปัญหาที่มีความซับซ้อนที่ยากที่สุดใน NP
- ปัญหาใด ๆ ในกลุ่มปัญหา *NP* สามารถเปลี่ยนรูป (Reduce) มาเป็นปัญหาใน **NP Complete** ได้
- เราจะเรียกปัญหาการตัดสินใจ *Q* ว่าเป็น เอ็นพีบริบูรณ์ เมื่อ
  1. *Q* เป็นปัญหา NP
  2. *Q* เป็นปัญหา NP- hard
     (นั่นก็คือทุกปัญหาใน NP สามารถเปลี่ยนรูปเป็น *Q* ได้)

เอกสารอ้างอิง

- NP-hard problem อ.สมชาย ประสิทธิ์จูตระกูล
- P and NP problem ดร.สุรศักดิ์ มังสิงห์
- http://th.wikipedia.org/wiki/%E0%B9%80%E0%B8%AD%E0%B9%87%E0%B8%99%E0%B8%9E%E0%B8%B5%E0%B8%9A%E0%B8%A3%E0%B8%B4%E0%B8%9A%E0%B8%B9%E0%B8%A3%E0%B8%93%E0%B9%8C#/media/File:P_np_np-complete_np-hard.svg